Using Database Transactions Can Help… Sometimes

If you are experienced with SQL-based databases, you may be wondering where database transactions fit in. If you aren't, a database transaction is a useful tool related to idempotence. A database transaction allows you to make many changes to a database and then apply them all at once. If anything should go wrong attempting to apply the changes, *none* of the changes are made. It's extremely handy, though it can't help as often as you might like and can have unintended performance consequences if you aren't careful.

As an example of how it works, suppose we want to reduce the hypothetical `quantity_remaining` of the product being ordered. You might do this by writing this code:

```
product.update!(quantity_remaining: product.quantity_remaining - 1)
order.update!(charge_id: payments_response.charge_id,
              charge_completed_at: Time.zone.now,
              charge_successful: true)
```

If something went wrong during the call to `order.update!`, the `quantity_remaining` would've been decremented, but the order not marked as charged. Upon retrying this code, the product quantity would be one less than the actual quantity after the order was fulfilled.

If, instead, you wrapped this code inside a call to `ActiveRecord::Base.transaction`, that failure could be eliminated:

```
ActiveRecord::Base.transaction do
  product.update!(quantity_remaining: product.quantity_remaining - 1)
  order.update!(charge_id: payments_response.charge_id,
                charge_completed_at: Time.zone.now,
                charge_successful: true)
end
```